

Skill 7

Escape Characters and Process Execution

Aim

To implement escape-character processing, handle escaped spaces and special symbols, preserve escaped characters, create child processes, execute programs, pass arguments and manage execution errors.

Requirements

- Ubuntu Linux
- GCC
- Terminal

Step-by-Step Implementation

Step 1: Understand Escape Characters

The backslash \ is used to escape special characters.

Example:

```
hello\ world
```

is interpreted as:

```
hello world
```

rather than two separate tokens.

Step 2: Handle Escaped Spaces

Input:

```
echo hello\ world
```

should produce:

```
hello world
```

as a single argument.

Step 3: Handle Special Symbols

Characters such as:

```
$  
&  
|  
>  
<
```

may require escaping when they need to be treated literally.

Step 4: Validate Parser Output

The parser should preserve escaped characters and generate the correct token sequence.

Step 5: Create Child Process

```
pid_t pid = fork();
```

Step 6: Execute the Program

Inside the child process:

```
execlp("ls", "ls", "-l", NULL);
```

Step 7: Handle Execution Errors

If execution fails:

```
perror("exec failed");  
exit(EXIT_FAILURE);
```

Step 8: Pass Arguments

For example:

```
ls -l /home
```

is passed as separate arguments to the executable.

Step 9: Manage the Parent Process

The parent waits for the child process to finish using:

```
waitpid(pid, NULL, 0);
```

Observation

Escape processing allowed spaces and special symbols to be preserved as literal characters. A child process was created using `fork( )` and the requested program was executed using the `exec( )` family. Execution errors were handled appropriately.

End Result

Escape-character handling and command execution using child processes were successfully implemented and tested.